About

The goal of this un-graded assignment is to provide a quick tutorial on how to download, compile, and execute NORI, the rendering framework that will be used during the course. Step-by-step instructions will be given for setting up the project, and on implementing a first `integrator` to generate a first image with the software.

NORI is a minimalistic ray tracer written in C++ by Prof. Wenzel Jakob (EPFL, Switzerland). It runs on Windows, Linux, and Mac OS and provides a foundation for the programming assignments in the Modeling and Simulation of Appearance course. While NORI provides much support code to simplify your development work as much as possible, the code that you will initially receive from us does very little: It loads a scene and saves a rendered image as both a PNG and OpenEXR image, but the actual rendering code is missing, hence the output image just consists of black pixels. Your task will be to extend this system to a full-fledged physically-based renderer as part of the lab assignments and the course project.

1 Nori core features

The NORI base code provides many features that would be tedious to implement from scratch. The following are included:

- A simple GUI to watch images as they render
- An XML-based scene file loader
- Basic point/vector/normal/ray/bounding box classes
- A pseudorandom number generator (PCG32)
- Support for saving output as OpenEXR files
- A loader for Wavefront OBJ files
- Support for textures in all materials
- An optimized bounding volume hierarchy builder
- Ray-triangle intersection

- Code for multi-threaded rendering
- Image reconstruction filters

2 References

You may find the following general references useful:

- *"Physically Based Rendering, Third Edition: From Theory To Implementation"*[1] by Matt Pharr, Wenzel Jakob, and Greg Humphreys. Morgan Kaufmann, 3rd edition, Nov 2016. http://www.pbr-book.org/
- *"Robust Monte Carlo Methods for Light Transport Simulation"*[2], PhD Thesis by Eric Veach, Stanford University, December 1997. https://graphics.stanford.edu/papers/veach_thesis/thesis-bw.pdf
- *"Advanced Global Illumination"* by Philip Dutre, Philippe Bekaert, and Kavita Bala. AK Peters, 2nd edition, August 2006.
- *"Global Illumination Compendium - The Concise Guide to Global Illumination Algorithms"*, Philip Dutre, 2003. https://people.cs.kuleuven.be/~philip.dutre/GI/

Feel free to consult additional references when completing projects, but remember to cite them in your writeup. When asked to implement feature X, we request that you don't go and read the source code of the implementation of X in some other renderer, because you will likely not learn much in the process. The PBRT book is excluded from this rule. If in doubt, get in touch with the course professors.

3 Instructions & Guidelines

There will be a number of programming assignments throughout the semester where you will build up specific functionalities in your rendering framework, followed by a course project at the end of the semester. You will need to work and submit the assignments and course project in groups of **maximum 2 people**. More information about the course project and the rendering contest will be provided halfway through the semester. The deadlines will be specified individually for each assignment, and they are different for each group. These deadlines strictly define the dates by which you need to submit the results of the assignments through **Moodle**.

The goal of the assignments is to help you create the foundations of a good rendering

[1] Scientific and Technical Academy Award
[2] Scientific and Technical Academy Award

framework for subsequent assignments and for the course project. It is your job to prove that you have implemented the assignments correctly through the results you will be submitting, and the quality and readability of your code. We will deduct points based on errors, artifacts, low-quality renders, and lack of comparisons. Make sure to start working on the homework as early as possible. Building your own advanced renderer is a lot of work, but also a lot of fun.

In the following, we will give you some detailed instructions on how to setup NORI (Section 4), we will guide you through the code and the scene definition in NORI (Section 5), and we will end up creating very first simple renderer in NORI (Section 7).

4 Downloading and setting up Nori

The base code of NORI that we will use during the course is provided as a `zip` file through Moodle. In the following, we describe how to set up the build system for each target platform using the CMake[3] build system. Tip: it's a good idea to set the build mode to Release unless you are tracking down a particular bug. The debug version runs much slower (by a factor of 50 or more).

> **CMake Troubleshooting:** After any error with CMake (e.g., because you were missing a library), remember to **remove CMakeCache.txt and CMakeFiles** in the build directory. Otherwise, CMake will keep failing even if you fix the problem.

Linux & Mac OS X Begin by installing CMake on your system. On Mac OS X, you will also need to install a reasonably up-to-date version of XCode along with the command line tools. On Linux, most versions of GCC or Clang will work (except g++13). Navigate to the NORI folder, create a build directory and generate the CMake files. You can use either the command line or the GUI:

1. **Using the command line (recommended!):**

```
$ sudo apt install g++-12 cmake
$ cd path-to-nori
$ mkdir build
$ cd build
$ cmake -DCMAKE_CXX-COMPILER=/usr/bin/g++-12 ..
```

2. **Using CMake-GUI:**

```
$ cd path-to-nori
$ mkdir build
$ cd build
```

[3] https://cmake.org/download/

```
$ cmake-gui ..
```

After the Makefiles are generated, simply run `make` (from the *build* directory) to compile all dependencies and NORI itself.

```
$ make -j 4
```

This can take quite a while; the above command compiles with four processors at the same time. Note that you will probably see many warning messages while the dependencies are compiled–you can ignore them.

Windows / Visual Studio Begin by installing Visual Studio (VS) 2017 or newer (older versions won't do) and a reasonably recent ($\geq$3.2) version of CMake. Community versions of VS should work just fine.

You can use either the command line or the GUI:

1. Using the command line (recommended):

Open a Powershell and navigate to the location where you extracted NORI. Generate the build files:

```
$ cd path-to-nori
$ cmake -G "Visual Studio 17 2022" -A x64 -B build
```

Then, each time you want to recompile run the following command from the nori base path (equivalent to make for Linux):

```
$ cmake --build build --config Release -j 4
```

2. Using CMake-GUI:

Start CMake and navigate to the location where you extracted NORI. It is a good idea to choose a `build` directory that is different from the source directory. After setting up the project, click the *Configure* and *Generate* buttons. This will create a file called `nori.sln` in the build directory – double-click it to open Visual Studio. The *Build→Build Solution* menu item will automatically compile all dependency libraries and Nori itself; the resulting executable is written to the Release or Debug subfolder of your chosen build directory. Note that you will probably see many warning messages while the dependencies are compiled – you can ignore them.

5 Nori's high-level overview

The NORI repository consists of the base code files and several dependency external libraries. Both are explained in Table 1. All your work will be implemented in the folders `src` and

include/nori. Take a moment to browse through the header files in include/nori. You will generally find all important interfaces and their documentation in this place. Most headers files also have a corresponding .cpp implementation file in the src directory. The most basic class in NORI is called NoriObject (include/nori/object.h) – it is the base class of everything that can be constructed using the XML scene description language (see Section 5.1). All other objects in NORI (e.g. cameras, geometry, materials, integrators...) derive from this class, and expose additional, more specific functionaliy. In addition, there are a few additional objects and headers that implement additional utility and mathematical functions, that will be used in the assignments.

All parameters of the scene being rendered, including emitters (light sources), geometry, and materials, are stored in the class Scene. It provides access to all elements in the scene, sampling rountines (that you will need to implement in the first assignment), and visibility computations by mean of *ray tracing*. Take a careful look on the interface of this class on include/nori/scene.h.

Disclaimer: Some of the concepts addressed in this script may not have been fully covered yet during the lectures (but will be, eventually). We recommend you keep this script at hand throughout the semester, and use this high-level overview as a reference.

Basic classes and structs NORI builds upon several classes that implement the basic elements of a renderer, in particular geometric data (i.e. points, vectors) and (spectral) radiance, as well as routines for sampling that you will be using during the next practical assignments. In particular, it is sensible to familiarize yourself with the following classes:

- **Vector3f, Point3f, Normal3f (include/nori/vector.h):** The basic geometric structures encode vectors, points and normals for N-dimensional spaces, implemented as templatized vectors from the linear algebra library Eigen. The actual structures we will be using are declared in include/nori/common.h. You can perform the basic scalar arithmetic operators over vectors, as well as accessing its elements using the operator Vector3f::operator[]. For more details, check the documentation of Eigen[4].
- **Frame (include/nori/frame.h):** Stores a three-dimensional orthonormal coordinate frame; This class is mostly used to quickly convert between different Cartesian coordinate systems and to efficiently compute certain quantities. Note that all shading operations in NORI are performed on a local orthonormal frame centered on the surface's normal.
- **Ray3f(include/nori/ray.h):** Simple ray-segment data structure used for ray tracing, as well as for querying radiance from the scene. This structure includes the origin and direction of the ray (members Ray::o and Ray::d respectively.

[4] https://eigen.tuxfamily.org/dox-3.2/

- **Color3f(include/nori/color.h):** Simple structure to store real-valued RGB radiance. Similar to e.g. `Vector3f` it derives from `Eigen`'s arrays, and allows element-wise operations in vectorial form.

- **Mesh (include/nori/mesh.h):** As most renderers out there, NORI supports triangle meshes for representing geometry. Essentially, each object is discretized as a bunch of triangles, which you can access to from the class `Mesh`. Additionally, it links the geometry with its particular `Material` and, in case of emitting surfaces, its `Emitter`. You can access the material and emitters by using the functions `Mesh::getBSDF()` and `Mesh::getEmitter()`, respectively. For more details, please see the interface source.

- **Intersection(include/nori/mesh.h):** Structure that stores the local information of an intersection between a `Ray` and 3D geometry in the `Scene` (modeled as triangle meshes using the class `Mesh`, see below). It includes all information required for shading the intersection point, including the `Point3f` of intersection, the local `Frame`, the texture coordinates, and a pointer to the associated `Mesh` from which the material can be accessed. To obtain the intersection between a `Ray` and the geometry in the scene, you should use the function `Scene::rayIntersect()` from the `Scene` (see details in `include/nori/scene.h`).

Exposed interfaces The base interfaces that derive from `NoriObject`, and that will describe both the implemented rendering algorithms and scene properties are:

- **Integrator (include/nori/integrator.h):** The different rendering techniques are collectively referred to as integrators, since they perform integration over a high-dimensional space. Each integrator represents a specific approach for solving the light transport equation—usually favored in certain scenarios, but at the same time affected by its own set of intrinsic limitations. During the next assignments, you will be asked to implement a few of these integrators. Integrators have one main function, `Integrator:Li()`, that for a given `Scene` and `Ray` generated from the camera, it returns a sample of the radiance carried by the ray, enconded in the class `Spectrum`.

- **Camera (include/nori/camera.h):** This class provides an abstract interface to cameras in Nori and exposes the ability to sample their response function by generating a ray to be traced towards the scene. By default, only the basic perspective (`src/perspective.cpp`) camera is implemented, but you can implement additional cameras in the course project.

- **Emitter (include/nori/emitter.h):** Superclass of all emitters (light sources) implemented in NORI. As all sampleable objects, it includes three main routines: a) `Emitter:sample()`, that samples a direction towards the emitter and stores it in a helper struct called `EmitterQueryRecord`; b) `Emitter:pdf()`, that returns the probability of generating a sample stored on an `EmitterQueryRecord`; and c) `Emitter:eval()`, that evaluates the light source for a sample stored in

Directory	Description
src	A directory containing the main C++ source code
include/nori	A directory containing header files with declarations
ext	External dependency libraries (see below)
scenes	Example scenes and test datasets to validate your implementation
CMakeLists.txt	A CMake build file which specifies how to compile and link Nori
External dependencies	
Directory	**Description**
ext/openexr	A high dynamic range image format library
ext/pcg32	A tiny self-contained pseudorandom number generator
ext/filesystem	A tiny self-contained library for manipulating paths on various platforms
ext/pugixml	A light-weight XML parsing library
ext/tbb	Intel's Boost Thread Building Blocks for multi-threading
ext/tinyformat	Type-safe C++11 version of printf and sprintf
ext/hypothesis	Functions for statistical hypothesis tests (won't be used in our assignments)
ext/nanogui	A minimalistic GUI library for OpenGL
ext/nanogui/ext/eigen	A linear algebra library used by `nanogui` and NORI.
ext/zlib	A compression library used by OpenEXR

Table 1: *Directory high-level description, and* NORI *dependencies.*

`EmitterQueryRecord`. More details on how to use `Emitter:sample()` will be found later in this assignment; while the rest will be used thoroughly in Assignment 2.

- **BSDF (`include/nori/bsdf.h`):** Superclass of all Bidirectional Scattering Distribution Functions, or BSDFs (appearance models) implemented in NORI. Similarly to `Emitter`, it is also a sampleable object and therefore implements three main routines: a) `BSDF:sample()`, that samples a direction and stores it in a helper struct called `BSDFQueryRecord`; b) `BSDF:pdf()`, that returns the probability of generating a sample stored on an `BSDFQueryRecord`; and c) `BSDF:eval()`, that evaluates the BSDF for a pair of directions stored in `BSDFQueryRecord`. More details on how to use `BSDF:eval()` will be found later in this assignment; the rest will be implemented and used in Assignment 2.

In Section 7 we show, step-by-step, how to implement your first physically-based `Integrator` for direct light (`DirectEmitterIntegrator`), as well as the most basic emitters (`PointLight`) and materials (`Diffuse`).

5.1 Scene file format and parsing

NORI uses a very simple XML-based description language to create scenes dynamically. The XML files are passed as a parameter to NORI to be parsed by the program, and can be regarded as a kind of building plan: NORI has an internal parser that creates scene elements step by step by reading the XML file from top to bottom. The XML tags in the XML document are interpreted as requests to create certain C++ objects, including information on how to put them together.

Each XML tag is either an object or a property. **Objects** correspond to C++ instances that will be allocated in memory. **Properties** are small bits of information that are passed to an object when the object is created. For instance, the following snippet creates red diffuse BSDF:

```
<bsdf type="diffuse">
   <color name="albedo" value="0.5, 0, 0"/>
</bsdf>
```

Here, the <bsdf> tag will cause the creation of an object of type BSDF, and the type attribute specifies what specific subclass of BSDF should be used. The <color> tag creates a property of name albedo that will be passed to its constructor. If you open up the C++ source file src/diffuse.cpp, you will see that there is a constructor, which looks for this specific property:

```
Diffuse(const PropertyList &propList) {
   m_albedo = propList.getColor("albedo", Color3f(0.5f));
}
```

The piece of code that associates the "diffuse" XML identifier with the Diffuse class in the C++ code is a macro found at the bottom of the file:

```
NORI_REGISTER_CLASS(Diffuse, "diffuse");
```

You will use this macro for all the new code you deploy using NORI (see Section 7).

Certain objects can be nested hierarchically. For example, the following XML snippet creates a geometry in the scene defined as a collection of triangles (i.e. a triangular mesh) using the <mesh> tag. The type obj within the <mesh> tag tells NORI that the specific positions of the triangles in the mesh have to be retrieved by reading the coordinates of their vertices from an external OBJ file; the path to the OBJ file is defined through the <string> tag, which is nested inside the <mesh></mesh> block. By nesting a <BSDF> block within the <mesh></mesh> block, we tell NORI to assign a diffuse BRDF to this specific mesh.

```
<mesh type="obj">
   <string type="filename" value="bunny.obj"/>

   <bsdf type="diffuse">
      <color name="albedo" value="0.5, 0, 0"/>
   </bsdf>
</mesh>
```

Implementation-wise, this kind of nesting will cause a method named addChild() to be invoked within the parent object. In this specific example, this means that Mesh::addChild() is called, which roughly looks as follows:

```
void Mesh::addChild(NoriObject *obj) {
   switch (obj->getClassType()) {
```

```
        case EBSDF:
            if (m_bsdf)
                throw NoriException(
                    "Mesh: multiple BSDFs are not allowed!");
            /// Store pointer to BSDF in local instance
            m_bsdf = static_cast<BSDF *>(obj);
            break;
    // ..(omitted)..
}
```

This function verifies that the nested object is a BSDF, and that no BSDF was specified before; otherwise, it throws an exception of type NoriException. Exceptions code blocks are commonly found throughout Nori, and their purpose is often to stop the execution, and warn the user about some erroneous or unsupported behavior in the execution of the program by showing a message in the command window.

The following different types of properties can currently be passed to objects within the XML description language:

```
<!-- Basic parameter types -->
<string name="property name" value="arbitrary string"/>
<boolean name="property name" value="true/false"/>
<float name="property name" value="float value"/>
<integer name="property name" value="integer value"/>
<vector name="property name" value="x, y, z"/>
<point name="property name" value="x, y, z"/>
<color name="property name" value="r, g, b"/>
```

```
<!-- Linear transformations use a different syntax -->
<transform name="property name">
    <!-- Any sequence of the following operations: -->
    <translate value="x, y, z"/>
    <scale value="x, y, z"/>
    <rotate axis="x, y, z" angle="deg."/>
    <!-- Useful for cameras and spot lights: -->
    <lookat origin="x,y,z" target="x,y,z" up="x,y,z"/>
</transform>
```

6 Visualizing HDR renders

High-dynamic-range images As we saw in the lectures (Introduction slides), most renderers, including NORI, generate high-dynamic-range (HDR) images. Specifically, NORI outputs images in EXR format (one of the standards for encoding HDR images). In HDR images, each of the RGB channels of each pixel is a floating-point number, while low-dynamic-range

(LDR) image formats (e.g. PNG, JPG, BMP) typically store pixels as 8-bit integers with only 256 possible values between 0 and 255. Most image viewers in your system or the web expect LDR images, and are not able to visualize HDR images, because the floating point values need to first be converted to LDR to be displayed by (most) screens. This process consists of the following two steps (also implemented in the Python snippet below):

1. *Tone mapping*, which reduces the HDR range from $[0, \text{float_max})$ to $[0, 1]$ (then discretized to an 8-bit integer value between 0 and 255).

2. *Gamma correction*, to better adapt to our non-linear perception of brightness.

Why doesn't Nori just output LDR images then? Because tone mapping and gamma correction throw away a lot of radiance information that might be useful for post-processing. For example, if you want to adjust the exposure of an image, or apply a different tone mapping operator, you would need to re-render the scene. Preserving HDR information of the rendered images allows you to to compare them to reference images to find energy mismatches, errors, and artifacts.

You could use a few different programs for visualizing EXR files (such as `Adobe Photoshop`, `HDRITools`[5] by Edgar Velázquez-Armendáriz, or `HDRView`[6] by Prof. Wojciech Jarosz). You could even make your own scripts with Python. If you are interested, you could start with the following snippet that implements EXR loading and the HDR-LDR conversion:

```python
import os, sys
os.environ["OPENCV_IO_ENABLE_OPENEXR"] = "1"
import cv2, numpy as np

def tonemap(img, exposure=0.0, gamma=2.2):
    img = img * (2.0 ** exposure) # exposure
    img = img / (1 + img) # reinhard tone mapping
    img = np.clip(img, 0, 1)
    img = img ** (1.0/gamma) # gamma correction
    return (img*255).astype(np.uint8) # convert to 8-bit

# Load HDR image:
img_hdr = cv2.imread(sys.argv[1], cv2.IMREAD_UNCHANGED)[:, :, :3]
img_hdr = cv2.cvtColor(img_hdr, cv2.COLOR_BGR2RGB)

# Convert to LDR:
img_ldr = tonemap(img_hdr, 0.0, 2.2, True)
```

However, for this course we *strongly recommend* using `TEV`[7]. `TEV` is a software that provides many useful features for visualizing and comparing HDR and LDR images, such as the ones you will be generating with NORI.

[5] https://bitbucket.org/edgarv/hdritools/downloads/
[6] https://bitbucket.org/wkjarosz/hdrview
[7] https://github.com/Tom94/tev

6.1 TEV: installation and basic usage

Installing `TEV` is straightforward in Windows, Linux and macOS. Install it from its repo: `https://github.com/Tom94/tev/#installation`. For Windows, we recommend the installer rather than the executable.

Once installed, you can use it by calling `tev` from the command line, or by setting it as the default program to open `.exr` files in your system.

Basic usage:

- `tev path-to-image.exr` opens an image
- `tev path/*.exr` opens all EXR files in a folder
- Once running, you can also open more images by dragging and dropping them into the window
- Click '?' for a complete list of keyboard shortcuts
- Once loaded, select an image by left clicking it in the bottom right. Then, the top left sliders control exposure and gamma.

Try loading any EXR (for example, this one[8] by Paul Debevec). After loading, you can select it by left-clicking it in the bottom right list. With an image selected, you have some useful controls:

- The top left sliders control exposure and gamma. The buttons below allow to change tone mapping operators. For example, "FC" is false color, useful for showing bright and dark colors.
- You can zoom in and see individual pixel values.
- There is a histogram view (middle left), displaying the distribution of pixel values. By hovering, it also shows the mean, minimum and maximum.

6.2 TEV: Comparing a render to a reference

Many times you will need to compare one of your renders to a reference we provide, or even to another of your own renders. This is easily done in TEV by selecting a loaded image by left clicking (it will be highlighted in blue) and then the reference by right clicking (highlighted in red). It will then display the per-pixel difference between both images.

Each of the "E" (Error), "AE" (Absolute Error), "SE" (Squared Error), "RAE" (Relative

[8] https://www.pauldebevec.com/Research/HDR/memorial.exr

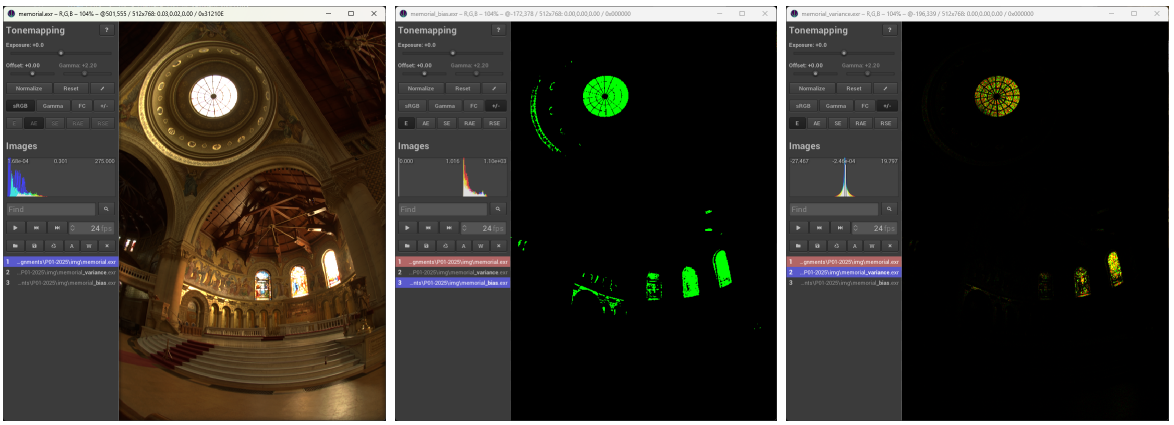


Figure 1: *Left:* memorial *image, the original (the other images* look *identical by default). Middle: comparison of* memorial-bias*, showing it is biased (green) in the windows. Right: comparison of* memorial-variance*, showing it has noise (both green and red). See how the histograms also show this, and experiment with other metrics and tonemappers.*

Absolute Error), "RSE" (Relative Squared Error) buttons will then select a metric to compare the two images. Hover the buttons to see the equation of the metric.

If one of the non-absolute metrics is selected, it might be more useful to select the "+/-" visualization mode above, which shows positive values in green and negative values in red. The histogram will also show the distribution of the metric, which can be very useful.

Example: Open all EXRs in the `memorial` folder. *memorial* is the original, while the *-bias* version has been edited to have much brighter windows and the *-variance* version has some random noise, also localized only in the windows. You will see that they look identical at first sight. This is only because of the tone mapping and the LDR display, which loses information.

How can you tell them apart? You will come accross this problem often when you compare your renders to our references, and it is important to distinguish them because images with bias are often caused by bugs, while variance is expected and usually not due to bugs.

The histogram of each image can help in some cases. In this example, you will see that the mean of the *-bias* image is much higher than the original. The *-variance* version is harder to distinguish, because the mean is not affected by the noise.

The comparison mode can help here. Select one of the modified images with left click, and the original with right click. Select the "E" (Error) metric and the "+/-" visualization mode. You will see that the *-bias* image has a very clear positive error (green) in the windows, while the *-variance* image has a noisy error, with both positive and negative values, as in Figure 1. Both comparison histograms also show this.

7 Creating your first Nori classes

In NORI, rendering algorithms that calculate the values of the pixels of an image are typically implemented as **integrators**, coded in children classes of the parent class `integrator`, and declared as `<integrator>` blocks in the XML files. They are so-called *integrators* because they generally solve different numerical *integration* problems; integrators typically calculate light intensity values (or any other scalar value) based on information about the scene. These values are eventually assigned to pixels of an image to be shown on a screen. The remainder of this section explains how to create your first integrators in NORI. We will start by creating a simple dummy integrator to visualize the *x,y,z* values of the surface normals of the scene as RGB pixel values. Then, we will create our first light transport-based integrator, that will allow us to visualize a scene as illuminated by **point light sources**.

7.1 NormalIntegrator

We begin by creating a new NORI object subclass in `src/normals.cpp` with the following content:

```
#include <nori/integrator.h>

NORI_NAMESPACE_BEGIN

class NormalIntegrator : public Integrator {
public:
    NormalIntegrator(const PropertyList &props) {
        m_myProperty = props.getString("myProperty");
        std::cout <<"Parameter value was : " <<m_myProperty <<std::endl;
    }

    /// Compute the radiance value for a given ray. Just return green here
    Color3f Li(const Scene *scene, Sampler *sampler, const Ray3f &ray) const {
        return Color3f(0, 1, 0);
    }

    /// Return a human-readable description for debugging purposes
    std::string toString() const {
        return tfm::format(
            "NormalIntegrator[\n"
            " myProperty = \"%s\"\n"
            "]",
            m_myProperty
        );
    }
protected:
    std::string m_myProperty;
};

NORI_REGISTER_CLASS(NormalIntegrator, "normals");
NORI_NAMESPACE_END
```

This integrator is so far very simple, since it only stores a `std::string` for debugging purposes, and renders all green. You can identify three main elements here: First, when creating any NORI object, you will pass to the constructor a list of properties stored in

`PropertyList`, which is created by the scene parser. You can search for properties using the functions `PropertyList::getXXX()` (see `include/nori/proplist.h` for all the getters). Then, the most important function is `Li`, that returns the color computed by the integrator (in this case, just green). Finally, the function `toString()` is shared by all NORI objects, and returns a human-readable description of the object. Note that in line 30 we include the `NORI_REGISTER_CLASS` macro, that links the class `NormalIntegrator` with the alias `normals` used in NORI's XML files.

To try out this integrator, we first need to add it to the CMake build system: For this, open `CMakeLists.txt` and look for the command

```
...
add_executable(nori,
  # Header files
  include/nori/bbox.h
  ...

  # Source code files
  src/bitmap.cpp
  ...
)
```

Add the line `src/normals.cpp` at the end of the source file list as:

```
...
add_executable(nori,
  # Header files
  include/nori/bbox.h
  ...

  # Source code files
  src/bitmap.cpp
  ...
  src/normals.cpp
)
```

You will need to do this for all classes you create in NORI. Note that in Windows you might need to re-run CMake for updating your VS solution. Recompile and if everything goes well, CMake will create an executable named `nori` (or `nori.exe` on Windows) which you can call on the command line.

Finally, create a small test scene with the following content and save it as test.xml:

```
<?xml version="1.0"?>

<scene>
   <integrator type="normals">
      <string name="myProperty" value="Hello!"/>
   </integrator>

   <camera type="perspective"/>
</scene>
```

This file instantiates our integrator and creates the default camera setup. Now, run NORI with this scene by introducing the following command in the console:

```
$ ./nori test.xml
```

Running NORI with this scene causes two things to happen: First, some text output should be visible on the console:

```
Property value was : Hello!

Configuration: Scene[
  integrator = NormalIntegrator[
    myProperty = "Hello!"
  ],
  sampler = Independent[sampleCount=1]
  camera = PerspectiveCamera[
    cameraToWorld = [1, 0, 0, 0;
                     0, 1, 0, 0;
                     0, 0, 1, 0;
                     0, 0, 0, 1],
    outputSize = [1280, 720],
    fov = 30.000000,
    clip = [0.000100, 10000.000000],
    rfilter = GaussianFilter[radius=2.000000, stddev=0.500000]

  ],
  medium = null,
  envEmitter = null,
  meshes = {
  }
]

Rendering .. done. (took 93.0ms)
Writing a 1280x720 OpenEXR file to "test.exr"
```

The NORI executable echoed the property value we provided, and it printed a brief human-readable summary of the scene, and a window with the rendered image pops up (in this case, a solid green window). Finally, **once the window is closed**, the rendered scene is saved as an OpenEXR file named `test.exr`, and a PNG image named `test.png`. In case you would like to render an image directly, without displaying it on a pop-up window, you can use the command `-b` or `--nogui` when calling NORI, as in the following example:

```
$ ./nori test.xml --nogui
```

Tracing rays

Let's now build a more interesting integrator which traces some rays against the scene geometry. Change the file `normals.cpp` as shown on below:

```
#include <nori/integrator.h>
#include <nori/scene.h>

NORI_NAMESPACE_BEGIN

class NormalIntegrator : public Integrator {
public:
    NormalIntegrator(const PropertyList &props) {
        // No parameters this time
    }

    Color3f Li(const Scene *scene, Sampler *sampler, const Ray3f &ray) const {
        // Find the surface that is visible in the requested direction
        Intersection its;
        if (!scene->rayIntersect(ray, its))
            return Color3f(0.0f);

        // Return the component-wise absolute
        // value of the shading normal as a color
        Normal3f n = its.shFrame.n.cwiseAbs();
        return Color3f(n.x(), n.y(), n.z());
    }

    std::string toString() const {
        return "NormalIntegrator[]";
    }
};

NORI_REGISTER_CLASS(NormalIntegrator, "normals");
NORI_NAMESPACE_END
```

In the code above, we intersect a ray with the scene by calling `scene→rayIntersect()` in line 15. This function returns true or false depending on whether an intersection exists, and if one does, it stores intersection information into the provided `Intersection` record (see Section 5. In the above example we return a color based on the surface normal at the hitpoint, which we compute by extracting the normal from the `Intersection`'s local `Frame` (note that for display, we compute the absolute value of the normal using `Normal::cwiseAbs()`). Also note that no parameter is read from the property list in the constructor, and that `NormalIntegrator::toString()` just displays the name of the integrator.

Invoking NORI on the file `scenes/assignment-1/bunny/bunny-normals.xml` as:

```
$ ./nori scenes/assignment-1/bunny-normals.xml
```

should produce the image shown in Figure 2, a shading normal rendering of the BUNNY scene.

7.2 Dot product Integrator

So far we have created a simple integrator that shows the normals of the object. We will now see how there there are several ways to render interesting images just with this normal information.

The `NDotLIntegrator` is a simple non-photorealistic rendering integrator that uses the

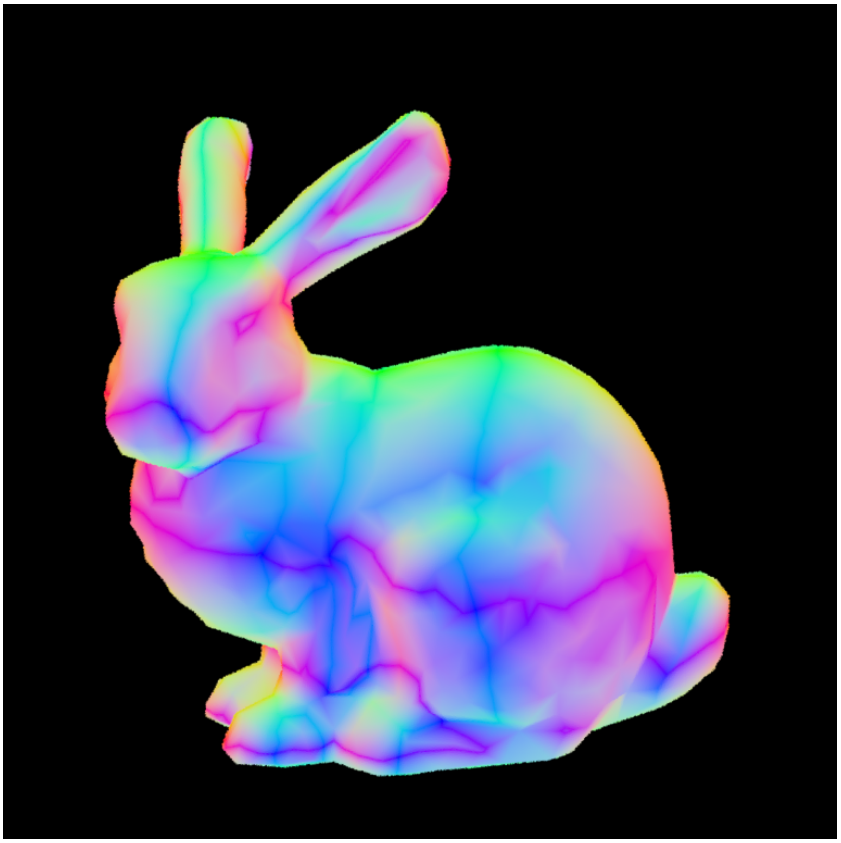


Figure 2: *A shading normal rendering of the* BUNNY *scene.*

dot product between the normal and a light direction to shade the image. Note that inputing the light direction and color as properties of the integrator is hacky (usually lights are separate objects in the scene), this exampple is just to show how you can add any information to integrators for debugging purposes.

```
#include <nori/integrator.h>
#include <nori/scene.h>

NORI_NAMESPACE_BEGIN

class NDotLIntegrator : public Integrator {
public:
    NDotLIntegrator(const PropertyList &props) {
        // Read the properties from the xml, or set some defaults:
        m_light_dir = props.getVector("light_direction", Vector3f(0.f,-1.f,0.f));
        m_light_dir = m_light_dir.normalized(); // direction vectors should be normalized
        m_light_color = props.getColor("light_color", Color3f(1.f));
    }

    Color3f Li(const Scene *scene, Sampler *sampler, const Ray3f &ray) const {
        // Find the surface that is visible in the requested direction
        Intersection its;
        if (!scene->rayIntersect(ray, its))
            return Color3f(0.0f);

        Normal3f n = its.shFrame.n; // get the normal from the intersection

        // dot(n, -light_dir) = how much the normal is aligned with the light direction
        Color3f color = m_light_color *std::max(0.f, n.dot(-m_light_dir));

        return color;
    }

    std::string toString() const {
        return "NDotLIntegrator[L=" + m_light_dir.toString() + "\n"
             + "\tlight_color=" + m_light_color.toString() + "\n]";
    }
protected:
    Vector3f m_light_dir;
    Color3f m_light_color;
};

NORI_REGISTER_CLASS(NDotLIntegrator, "NDotL");
NORI_NAMESPACE_END
```

Render the bunny with this integrator. For example, in windows:

```
$ build/Release/nori.exe scenes/assignment-01/bunny/bunny-ndotl.xml
```

This will produce images like 3 (left). You can modify the light direction and color by uncommenting lines 9 and 10 in the xml file (right):

This idea of manipulating normals is widely used in Non-Photorealistic Rendering (NPR). See the Appendix 8 if you are interested in an example of a toon integrator, where just the normals and light direction enables interesting renders.

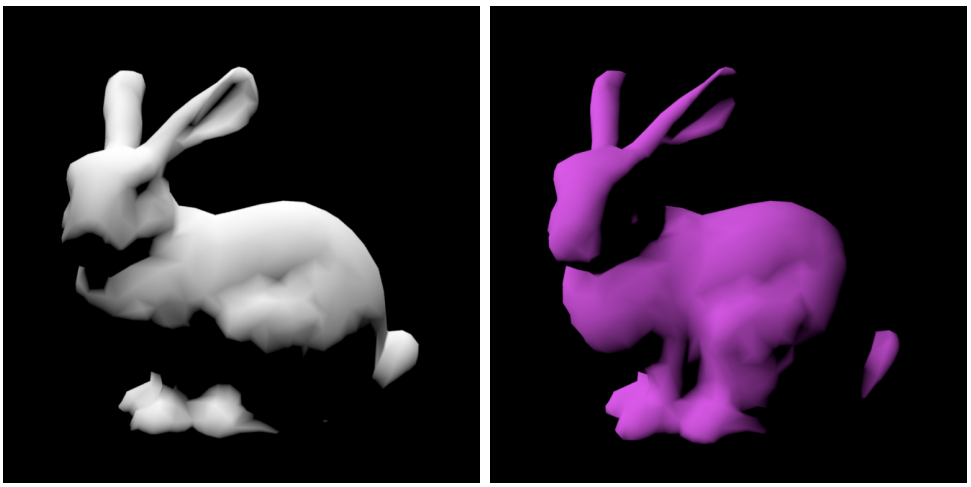


Figure 3: *A non-photorealistic rendering of the* BUNNY *scene using the NDotL integrator. Left: default xml. Right: modifying the parameters in the xml (uncomment the lines 9 and 10 in the xml)*

7.3 DirectWhittedIntegrator

Now, we will implement the first integrator that actually computes a (still simplified) light transport model: **direct light from point light sources**. For that, we will need to define a new type of light source (`PointLight`) and a new integrator (`DirectWhittedIntegrator`), while using the simple Lambertian material already implemented in `src/diffuse.cpp`. This first integrator will handle direct illumination with ray-traced shadows from point lights (the kind of scenes that Turner Whitted's ray tracing handled back in 1978).

First of all, let's create two new files in `src/` called `pointlight.cpp` and `direct_whitted.cpp`, and add them to the CMake build system, opening `CMakeLists.txt` and adding two additional lines:

```
...
add_executable(nori,
  # Header files
  include/nori/bbox.h
  ...

  # Source code files
  src/bitmap.cpp
  ...
  src/normals.cpp
  src/pointlight.cpp
  src/direct_whitted.cpp
)
```

Re-run CMake to include it in the make system. Now it is time for coding both sources. Let us start with `pointlight.cpp`, which will implement a `PointEmitted`, and therefore will inherit from `Emitter`, as:

```
#include <nori/emitter.h>

NORI_NAMESPACE_BEGIN
```

```
class PointEmitter : public Emitter {
public:
  PointEmitter(const PropertyList &props) {
    m_type = EmitterType::EMITTER_POINT;
    m_position = props.getPoint("position", Point3f(0.,100.,0.));
    m_radiance = props.getColor("radiance", Color3f(1.f));
  }

  virtual std::string toString() const {
    return tfm::format(
      "PointEmitter[\n"
      " position = %s,\n"
      " radiance = %s,\n"
      "]",
      m_position.toString(),
      m_radiance.toString());
  }

  virtual Color3f eval(const EmitterQueryRecord & lRec) const {
    // This function assumes that a ray have been traced towards
    // the light source. However, since the probability of randomly
    // sampling a point in space is 0, its evaluation returns 0.
    return 0.;
  }
  virtual Color3f sample(EmitterQueryRecord & lRec,
                         const Point2f & sample,
                         float optional_u) const {

    lRec.p = m_position;
    lRec.dist = (lRec.p -lRec.ref).norm();
    lRec.wi = (lRec.p -lRec.ref) / lRec.dist;

    // Note that the pdf should be infinite, but for numerical
    // reasons it is more convenient to just leave as 1
    lRec.pdf = 1.;

    // Note that here it is assumed perfect visibility; this means
    // that visibility should be taken care of in the integrator.
    return m_radiance/(lRec.dist*lRec.dist);
  }

  // Note that the pdf should be infinite, but for numerical reasons
  // it is more convenient to just leave as 1
  virtual float pdf(const EmitterQueryRecord &lRec) const {
    return 1.;
  }


protected:
  Point3f m_position;
  Color3f m_radiance;
};

NORI_REGISTER_CLASS(PointEmitter, "pointlight")
NORI_NAMESPACE_END
```

Essentially, we have implemented the `PointLight::sample()` function, that fills out an `EmitterQueryRecord` and returns the radiance emitted by the light in point `lRec.p` as a `Color3f`. The remaining functions (`eval()` and `pdf()`) do not have utility for singular light sources, as we will explain later in the course (essentially, a point light is a singularity in space, and therefore cannot be intersected by a random ray).

Once we have a light source to light the scene, we move to `direct_whitted.cpp`, where we will implement `DirectWhittedIntegrator`, a simple integrator with no parameters nor preprocessing, so that the only function that matters is `Li()`. This is the code for implementing `DirectWhittedIntegrator`. **Note that some parts are missing, as you will have to fill them out:**

```
#include <nori/warp.h>
#include <nori/integrator.h>
#include <nori/scene.h>
#include <nori/emitter.h>
#include <nori/bsdf.h>

NORI_NAMESPACE_BEGIN

class DirectWhittedIntegrator: public Integrator {
public:
  DirectWhittedIntegrator(const PropertyList& props) {
    // No parameters this time
  }

  Color3f Li(const Scene* scene, Sampler* sampler, const Ray3f& ray) const {

    Color3f Lo(0.);

    // Find the surface that is visible in the requested direction
    Intersection its;
    if (!scene->rayIntersect(ray, its))
      return scene->getBackground(ray);

    float pdflight;
    EmitterQueryRecord emitterRecord(its.p);

    // Get all lights in the scene
    const std::vector<Emitter*> lights = scene->getLights();

    // Let's iterate over all emitters
    for (unsigned int i = 0; i <lights.size(); ++i)
    {
      const Emitter* em = lights[i];

      // Here we sample the point sources, getting its radiance
      // and direction.
      Color3f Le = em->sample(emitterRecord, sampler->next2D(), 0.);

      // Here perform a visibility query, to check whether the light
      // source "em" is visible from the intersection point.
      // For that, we create a ray object (shadow ray),
      // and compute the intersection
      // --------------------------------
      // YOUR CODE HERE
      // --------------------------------

      // Finally, we evaluate the BSDF. For that, we need to build
      // a BSDFQueryRecord from the outgoing direction (the direction
      // of the primary ray, in ray.d), and the incoming direction
      // (the direction to the light source, in emitterRecord.wi).
      // Note that: a) the BSDF assumes directions in the local frame
      // of reference; and b) that both the incoming and outgoing
      // directions are assumed to start from the intersection point.
      BSDFQueryRecord bsdfRecord(its.toLocal(-ray.d),
                         its.toLocal(emitterRecord.wi), its.uv, ESolidAngle);

      // For each light, we accomulate the incident light times the
      // foreshortening times the BSDF term (i.e. the render equation).
      Lo += Le *its.shFrame.n.dot(emitterRecord.wi) *
```

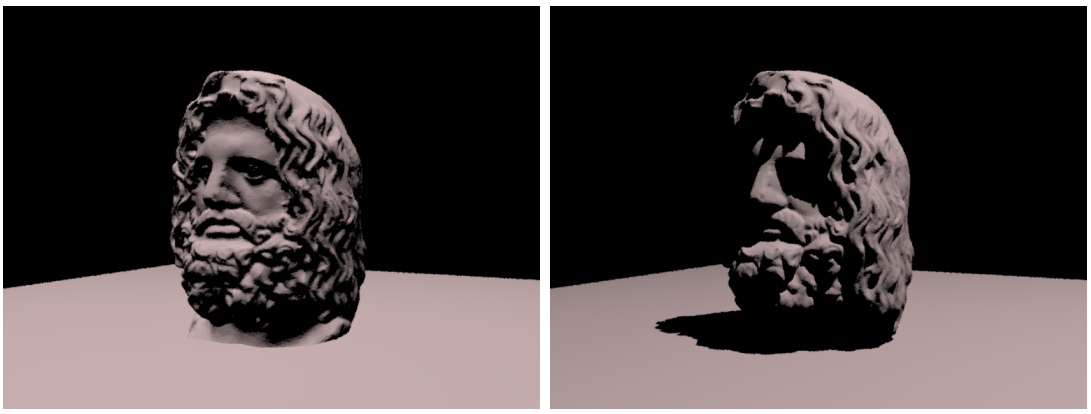


Figure 4: *The* SERAPIS *scene rendered with direct illumination and a single point light source. Left: original code that does not check for shadows. Right: modified code to account for shadows.*

```
                        its.mesh->getBSDF()->eval(bsdfRecord);
    }

    return Lo;
  }

  std::string toString() const {
    return "Direct Whitted Integrator []";
  }
};

NORI_REGISTER_CLASS(DirectWhittedIntegrator, "direct_whitted");
NORI_NAMESPACE_END
```

What `DirectWhittedIntegrator::Li()` does is to compute the radiance from `Ray ray` by going through all light sources in the scene (lines 27-31), accomulating their contribution according to the rendering equation: First the amount of light from the light source is computed (line 37), as well as the direction from the light source to the illuminated point. Then, **you should modify the code to evaluate whether the point is in shadows or not (lines 39-47).** Finally (lines 49-62), if the point is not in shadow, compute the amount of light reflected by multiplying the incoming radiance in `Le` with the dot product term modeling foreshortening and the evaluation of the surface's BSDF (which you can access by using `its.mesh->getBSDF()`).

Invoking NORI on the file `scenes/assignment-1/serapis-whitted.xml` as:

```
$ ./nori scenes/assignment-1/serapis-whitted.xml
```

should produce the image shown in Figure 4, a direct light rendering of the SERAPIS scene illuminated by a single point light source.

7.4 AOV Integrator

AOV (Arbitrary Output Variables) are extra images that can be rendered, encoding a specific value per pixel. These variables could be anything: depth, normals, albedo, or any other value you may need for debugging. We will implement an integrator with some example variables. Remember this integrator: you can extend it along the course for your needs.

It will read a string called `variable` from the XML file, which will select the desired output.

```
#include <nori/integrator.h>
#include <nori/scene.h>

#include <nori/bsdf.h>

NORI_NAMESPACE_BEGIN

class AOVIntegrator : public Integrator {
public:
    AOVIntegrator(const PropertyList &props) {
        m_mode = props.getString("variable", "normals");
    }

    Color3f Li(const Scene *scene, Sampler *sampler, const Ray3f &ray) const {
        // Find the surface that is visible in the requested direction
        Intersection its;
        if (!scene->rayIntersect(ray, its))
            return Color3f(0.0f);

        if (m_mode == "normals") {
            Normal3f n = its.shFrame.n; // get the normal from the intersection
            // Map the normal from [-1,1] to [0,1] for visualization
            return .5f + .5f *Color3f(n.x(), n.y(), n.z());
        } else if (m_mode == "depth") {
            throw NoriException("AOVIntegrator: You need to implement the depth mode!");
        }
        else if (m_mode == "uv") {
            return Color3f(its.uv.x(), its.uv.y(), 0.f);
        }
        else if (m_mode == "albedo") {
            const BSDF *bsdf = its.mesh->getBSDF(); // get the material
            // Get the base color (albedo) at this point of the surface:
            Color3f albedo = bsdf->getAlbedo(its.uv);
            return albedo;
        }
        else {
            throw NoriException("AOVIntegrator: Unimplemented mode \"%s\"!", m_mode.c_str());
        }
    }

    std::string toString() const {
        return "AOVIntegrator[mode=" + m_mode + "]";
    }
protected:
    std::string m_mode;
};

NORI_REGISTER_CLASS(AOVIntegrator, "aov");
NORI_NAMESPACE_END
```

Figure 5 (left) shows an example of rendering the `cbox-aov` scene, which shows the albedo (base color) of each surface in the Cornell Box scene.

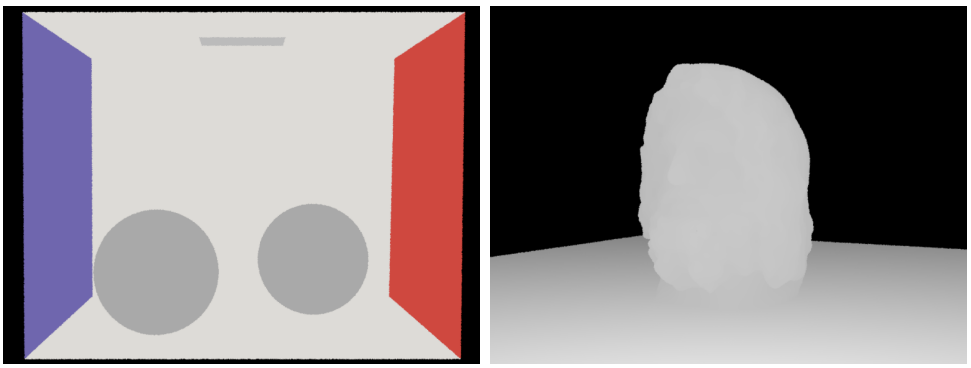


Figure 5: *Two example renders with the AOV Integrator. Left:* CBOX-AOV *scene, showing the albedo of each surface. Right: A render showing depth in the* SERAPIS *scene.*

Lastly, you will need to implement the depth mode: compute the distance of each point in the scene to the origin of the camera. You can implement this part however you prefer. After editing `serapis-whitted.xml` to use this integrator and selecting the depth variable, Figure 5 shows the expected output: each pixel represents $1/d$, where d is the distance from the camera origin to the first intersection point along the ray.

8 Bonus: Toon shading integrator

The following integrator `ToonIntegrator`[9], which you can find in `src/01_toon.cpp`, implements a simple toon shading effect by separating the NDotL value into two tones, and finding specular highlights and the edges for rim lighting.

```
Color3f Li(const Scene *scene, Sampler *sampler, const Ray3f &ray) const {
    // Find the surface that is visible in the requested direction
    Intersection its;
    if (!scene->rayIntersect(ray, its))
        return Color3f(0.0f);

    Normal3f n = its.shFrame.n; // get the normal from the intersection

    float ndotl = n.dot(-m_light_dir); // cosine with the light direction

    // toon makes it either fully lit or fully shadowed:
    float light = smoothstep(0.f,0.05f, ndotl); // with a smooth but still sharp transition

    // Specular highlights.
    // Find how much the normal is aligned with the half vector towards the light
    Vector3f half = (-ray.d -m_light_dir).normalized();
    float ndoth = n.dot(half);

    float specular_highlight = pow(ndoth *light, 80.f); // vectors are very aligned
    specular_highlight = smoothstep(0.8f, 0.9f, specular_highlight); //line instead gradient

    // Edges are the places where the normal is close to perpendicular to the view direction:
    float edge = n.dot(ray.d);
    edge = pow(edge*edge, 2.f); // 0 only when the vectors are very perpendicular
```

[9] Adapted from this blog by Roystan: https://roystan.net/articles/toon-shader/



Figure 6: *Toon renderings of the* BUNNY *scene using the Toon integrator. Left: Before modifying the code. Middle: rim lighting and highlights enabled by removing some 0s in the toon code. Right:* `serapis-toon`.

```
    edge = smoothstep(0.005f, 0.01f, edge); // make it a line instead of a gradient
    edge = 1.f -edge; // invert it, so that edges are 1 and other places are 0
    edge = edge *light; // only show edges on the lit side

    // Combine light, specular highlights, edges and some ambient color:
    Color3f color = m_light_color*(light+specular_highlight+edge) + m_ambient_color;
    return color;
}
```

You will not need to use these kind of integrators in the assigments, but it can be useful to get familiar with some of the geometry involved. You can try removing some of the components of this integrator (highlights, edges, etc.) to see how they affect the final image. You can also set its properties in `scenes/assignment-01/bunny/bunny-toon.xml`, and run it as:

```
$ build/Release/nori.exe scenes/assignment-01/bunny/bunny-toon.xml
```

To produce images like the ones in Figure 6. Bonus: add shadows to this integrator (it should be around 2 lines of code).

9 What to submit?

While this assignment is **not evaluable**, to check the completion of the task you should submit the `exr` and `png` image outputs of the `scenes/assignment-1/bunny-normals.xml`, `scenes/assignment-1/serapis-whitted.xml`, and of the modified `serapis-whitted.xml` that uses the AOV integrator. Zip all images in a single `zip` file with name "`p1_NIP1_NIP2.zip`", and submitted in Moodle as a response of the assignment, with NIP1 and NIP2 the university ID numbers of each member of the team (remember, max two people per team!).

You will find different submission entries for each group in Moodle. **Make sure you submit to the one corresponding to your group.**